waLLMartCache: A Distributed, Multi-Tenant and Enhanced Semantic Caching System for LLMs

Soumik Dasgupta, Anurag Wagh, Lalitdutt Parsai, Binay Gupta, Geet Vudata, Shally Sangal, Sohom Majumdar, Hema Rajesh, Kunal Banerjee[0000−0002−0605−630X]⋆, and Anirban Chatterjee

Walmart Global Tech, Bangalore, India
{soumik.dasgupta, anurag.wagh, lalitdutt.parsai, binay.gupta, geet.vudata, shally.sangal, sohom.majumdar, hema.rajesh, kunal.banerjee1, anirban.chatterjee}@walmart.com

Abstract. In recent times, Large Language Models (LLMs) have become an important tool for various business use cases. The power of these LLMs can be leveraged in improving developer productivity, as a knowledge base, for creating applications such as question answering and information retrieval systems. Unfortunately, these benefits come at the price of high usage costs and latency. The managed LLM services charge based on tokens (words) processed which become very significant with scale. Even self-hosted open source LLMs turn out to be expensive because hosting requires expensive GPUs and scaling to many requests would need significant horizontal scaling of these resources. In this context, an LLM focused caching system can significantly reduce usage costs as well as latency. This problem is addressed by GPTCache. The current work termed waLLMartCache advances GPTCache by incorporating the following features: (i) we introduce the support for a new database Redis in GPTCache (our pull request is already merged with GPTCache main branch) – this is used as L2 cache in our designed system, (ii) presently, GPTCache is implemented to be run on a single node which we enhance to span across multiple nodes to handle industry-scale requests and consequently, we also designed a distributed eviction manager, (iii) we further create partitions for individual tenants (clients) so that these can be hosted together while maintaining semantic separations, (iv) we present a *decision engine* that decides whether to cache an LLM response based on our business use-cases, and (v) we showcase that loading FAQs (which can be set to be stored persistently in the memory) while booting the LLM cache can be a simple yet effective strategy to boost cache hits significantly. Although this system is in-house to our company, we believe that the methodology shared in this paper is generic enough to be adopted by any organization.

Keywords: LLM · GPTCache · Redis · Distributed system · Eviction manager.

⋆ Corresponding Author

1 Introduction

Large Language Model (LLM) is considered as one of the biggest disruptors in technology nowadays. Although language models have persisted for many years [7], LLMs have become immensely popular since the release of GPT-3 [9] by OpenAI that later caught more attention with the release of GPT-3.5 [32] and GPT-4 [21]. These LLMs have been found to be good at various tasks, such as answering questions, summarizing texts, synthesizing new content, etc. while newer applications involving the LLMs are being invented almost every day. However, accessing the GPTs can be quite costly [20]; moreover, accessing them using APIs typically involves non-uniform latencies – the situation is further exacerbated if too many requests come from a user and consequently, the service is temporarily revoked for that user; in such cases, the service is resumed only after some cool-down period. Such untoward events hamper developer productivity and customer experience significantly which, in turn, may lead to revenue and/or reputation loss for businesses that are dependent on the GPTs.

To alleviate these problems, GPTCache was introduced in [5]. GPTCache, similar to the concept of the standard caches, stores responses from previous queries sent to the LLMs and whenever a *"similar"* query is posed again, the response is returned from the cache instead of invoking the LLM again. Thus, GPTCache not only saves the cost incurred by availing the services from the GPTs but also reduces the response speed by up to 10x [5]. Moreover, returning the response from the cache is unaffected by network fluctuations – thereby making the system more stable; inadvertent lockout periods from the GPT services are also reduced if not avoided altogether.

Notably, GPTCache, in contrast to what its name may suggest, is applicable to any LLM such as, Gemini [3], Claude [4], Llama2 [28], Falcon [2], Mistral [16] and Dolly [12] just to name a few. Although GPTCache's benefits are undoubtedly great, adopting it off-the-shelf for industry use-cases posed many challenges, especially when it came to scaling the cache to a distributed system to handle thousands of requests per minute. Consequently, we had to make upgrades to the GPTCache which resulted into its updated version that we termed as *waLLMartCache*. Our system waLLMartCache advances GPTCache by incorporating the following features:

1. We introduce the support for a new database Redis [27] in GPTCache (our pull request is already merged with GPTCache main branch[1]) – this is used as L2 cache in our designed system.
2. Currently, GPTCache is implemented to be run on a single node which we enhance to span across multiple nodes to handle industry-scale requests and consequently, we also designed a distributed eviction manager.
3. We further create partitions for individual tenants (clients) so that these can be hosted together while maintaining semantic separations.
4. We present a *decision engine* that decides whether to cache an LLM response based on retail business use-cases.

[1] https://github.com/zilliztech/GPTCache/releases/tag/0.1.35

5. We showcase that loading FAQs (which can be set to be stored persistently in the memory) while booting the LLM cache can be a simple yet effective strategy to boost cache hits significantly.

It is important to note that although waLLMartCache is in-house to our company, we believe that our methodology is generic enough to be adopted by any organization.

The paper is organized as follows. Section 2 covers the related literature. Section 3 describes GPTCache in brief. Section 4 explains our enhancements made on top of GPTCache along with some empirical evidence of waLLMartCache's efficacy. Section 5 concludes the paper with a summary of the work done and some potential future directions.

2 Related Work

Cache memory is a well-known concept in computer architecture whereby certain information (instructions and/or data) is stored in a designated memory that has a lower latency than the primary memory [15]. Semantic caching [11] stores data in cache based on its meaning in contrast to traditional caching techniques which typically exploit the concept of *locality of reference* [15] and does not usually look into the meaning of the data. The idea of semantic caching is utilized in [17] to convert cache misses to cache hits while accessing web sources through user queries. In another work [19], semantic caching is used in the context of database queries where a differentially private cache of earlier responses is employed to answer the current workload while maintaining a privacy budget.

GPTCache [5] presents an open-source semantic cache that is specifically geared towards providing faster LLM responses in addition to lowering the costs of LLM usage. We will cover GPTCache, which is the foundation for the current work, in detail in Section 3. GPTCache has already been integrated into applications such as, AI Tutor [6], social and e-commerce recommender [31], and power-aware model serving framework [25]. Note that LLM inference times vary based on the lengths of the generated responses, and since LLM serving systems generally serve requests on a First-Come-First-Serve (FCFS) basis, they often suffer from head-of-line blocking issues. The authors of [24] plan to explore GPTCache in conjunction with a speculative shortest-job-first scheduler to tackle this problem. The same FCFS related problem is also addressed by the authors of [10] who build a Language-Model-as-a-Service (LMaaS) based on efficient batching of the LLM queries which, in turn, relies on predicting the lengths of the generated responses. In the work described in [33], the authors propose a distillation-based method to fine-tune the existing embeddings for better caching prediction. A similar distillation based strategy is adopted in [1] and compared against GPTCache – however, it may be noted that this method specifically focuses on text-to-image generation prompts. In another related work [26], how to generate semantically similar prompts has been explored – these prompts

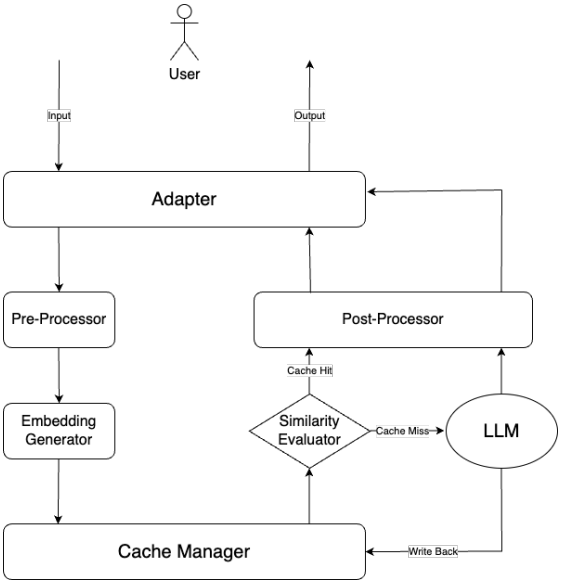

Fig. 1. The overall architecture of GPTCache. Note that in the original paper [5], the Post-Processor module is applied only in case of cache hits but we apply it for both cache hits and misses to adhere to the best industry practices.

should help in determining validity of cache hits and misses, and thus contribute to evaluating the efficacy of the semantic caches.

3 GPTCache in a Nutshell

In this section, we briefly describe GPTCache whose details can be found at [5]. Conceptually, GPTCache is similar to other caches – whenever a new query is fired, we first check if that query has already been posed to the LLM or not and if a match is found, then the response is returned immediately from the cache; otherwise, we follow the traditional route of sending the query to the LLM and returning its response to the user – however, additionally, the query and the response are stored in the cache for future references. Below are the modules that are present in GPTCache as shown in Fig. 1. While explaining the GPTCache, we also underline some salient features where our system design deviates from the original one even when the aspects of distributed system, multi-tenancy, etc. are not considered.

3.1 Adapter

This module serves as the interface for GPTCache that is accountable for converting LLM queries into cache protocols and converting cache results into LLM

responses. Basically, the user should experience identical input/output behavior from the system irrespective of whether caching is enabled or not; the adapter is charged with ensuring a smooth and uniform user experience.

3.2 Pre-Processor

This module removes some of the redundant information that may be present in the prompt, e.g., user identifiers. To understand an interesting example where pre-processing may be required, let us consider the case where a long chat is going on between the user and the LLM – however, every LLM typically has a restriction on the number of tokens that it can process in the input prompt, and hence upon nearing the maximum token limit, the pre-processing module may compress the initial conversations or drop them altogether based on the assumption that the recent chats influence the response considerably more than the older ones. Note that dropping early chats may at times incur some performance degradation compared to compression but on the other hand, it may have significant economic impact – in industry, one typically works with the individual clients to employ the best possible pre-processing strategy that suits their performance needs while reducing their costs.

3.3 Embedding Generator

This module translates user queries into embedding vectors which are later used for similarity based retrievals.

3.4 Cache Manager

This module lies at the core of GPTCache and primarily performs three subtasks.

- **L1 Cache Storage:** When a user query arrives, it is first converted into an embedding vector and stored in a vector database such as, Milvus [29], Pinecone [22], Qdrant [23] or Weaviate [30]. For each user query, along with an embedding vector, a unique scalar id is also generated that is used subsequently in the L2 cache.
- **L2 Cache Storage:** This storage stores the unique scalar ids generated by the L1 cache along with the corresponding LLM responses. Any database (including simple structural databases) that can store key-value pairs efficiently can be used as the L2 cache.
- **Eviction Management:** This sub-module clears the cache by following a pre-determined policy, for example, the Least Recently Used (LRU) policy or the First-In-First-Out (FIFO) policy, to maintain the cache capacity.

It may be worth noting that the original paper [5] uses the terms *vector storage* and *cache storage* whereas, we use *L1 cache storage* and *L2 cache storage*, respectively. We deviate from the original terminology because of two reasons: (i)

we found the terms *vector storage* and *cache storage* confusing because both are part of GPTCache, and (ii) L1 and L2 storages are commonly used terms in the context of caching and also imply the order in which these storages are accessed (similar to standard caches).

3.5 Similarity Evaluator

This module finds the most similar query posed earlier based on the embedding vectors generated. Typically, Euclidean distance or cosine similarity is used to find the most similar query; however, if the distance is found to be greater than some threshold d, then it is considered to be a *cache miss* and the LLM is actually invoked.

3.6 Post-Processor

This module massages the LLM response before it is outputted to the user. In an industry context, we apply *content filtering* to filter the texts that may be toxic or derogatory [13] which may lead to reputation loss. Furthermore, we also scan for *Personal Identifiable Information (PII)* data and redact these from the outputted responses to adhere to laws and regulations, wherever applicable. It may be noted that while in the original GPTCache, *post-processing* is applied only in the case of cache hits, we apply it both during cache hits and misses to ensure that content filtering and redacting PII data are applied (among other similar processes) so that AI is always used responsibly for both internal and external uses.

4 waLLMartCache: An Enhanced Cache for LLMs

In this section, we delve into the details of the significant enhancements which have been done to the GPTCache to adapt it for industry requirements.

4.1 Incorporating Redis as a Database

We had experimented with multiple databases before finalizing on Redis [27] as the L2 cache storage. The experimental results with Redis are captured in Table 1. The dataset used for these experiments is obtained from our in-house *Generative AI playground* where any internal associate can pose a query to an LLM and get its response. Although this *"playground"* is good for testing LLMs and not meant for productionization (which may involve use-case specific fine-tuning), we chose this dataset because it covered a large spectrum of the queries (being fired by associates from all the teams) that are of importance to the retail industry, and thus can act as a suitable proxy for our initial generic design choices. The queries present in the dataset typically range from 500 to 1000 tokens. The LLM responses, however, can be very large, and hence we bucketize these responses (referred to as *Content* in Table 1). As shown in Table 1, we

Table 1. Experimental results with Redis as an L2 cache storage

Content Size (#tokens)	#Concurrent Users	#Requests	#RPS	Average Response Time (ms)	Median Response Time (ms)	P90 Response Time (ms)
<2K	500	135758	156	119	45	110
	1000	170530	258	614	190	1100
2K to 5K	500	83035	155	141	55	140
	1000	180694	239	1051	180	3800
5K to 10K	500	81536	158	152	91	230
	1000	122708	217	1523	240	4800
>10K	500	91331	154	227	140	400
	1000	103840	180	2463	380	7400
All buckets	500	135365	150	173	48	120
	1000	190668	230	952	200	2100

also consider the case where contents from all the different buckets are present together and we refer to it as *All*. We check how much scale can Redis handle by deploying 500 and 1000 concurrent users at once. We report the total number of requests, Requests Per Second (RPS) and the average, the median and the P90 (90th percentile) response times for each combination of bucket size and number of concurrent users. Due to our internal non-competition policy, we refrain from mentioning the other databases that we explored. Nevertheless, it may be noted that *the closest competition scaled to only 30% of the RPS that was registered for Redis for the same configuration.* Lastly, our code for supporting Redis in GPTCache is already merged with the open source repository and available for everyone to use.

4.2 Designing a Distributed Cache for LLMs

This enhancement lies at the core of our designed system. We elaborate it with the help of Fig. 2. In case of an industry-scale application, several hundreds of requests may be submitted every second by hundreds of customers and associates. The queries coming from the various users first undergo a load balancer that distributes these queries uniformly across various nodes, each running an instance of our waLLMartCache. After passing through the Adapter, a query comes to the Distributed Cache Manager that has a local L1 Cache Storage Manager and a local L2 Cache Storage Manager. There is no local storage residing in these individual nodes. Once the semantic embedding is obtained for the query, we search for its most similar query in the central L1 Cache Storage. If no similar query is found, then the LLM is invoked and the response is returned to the user along with saving a copy in the L2 Cache Storage. In contrast, if a similar query is found, then the L1 Cache Storage sends the corresponding scalar id to the L1 Cache Storage Manager in the originating waLLMartCache node, which shares the same to the L2 Cache Storage Manager that fetches the saved LLM response from the central L2 Cache Storage that is finally returned to the user. When any of the storages nears its capacity limit, we evict the entries that are deemed

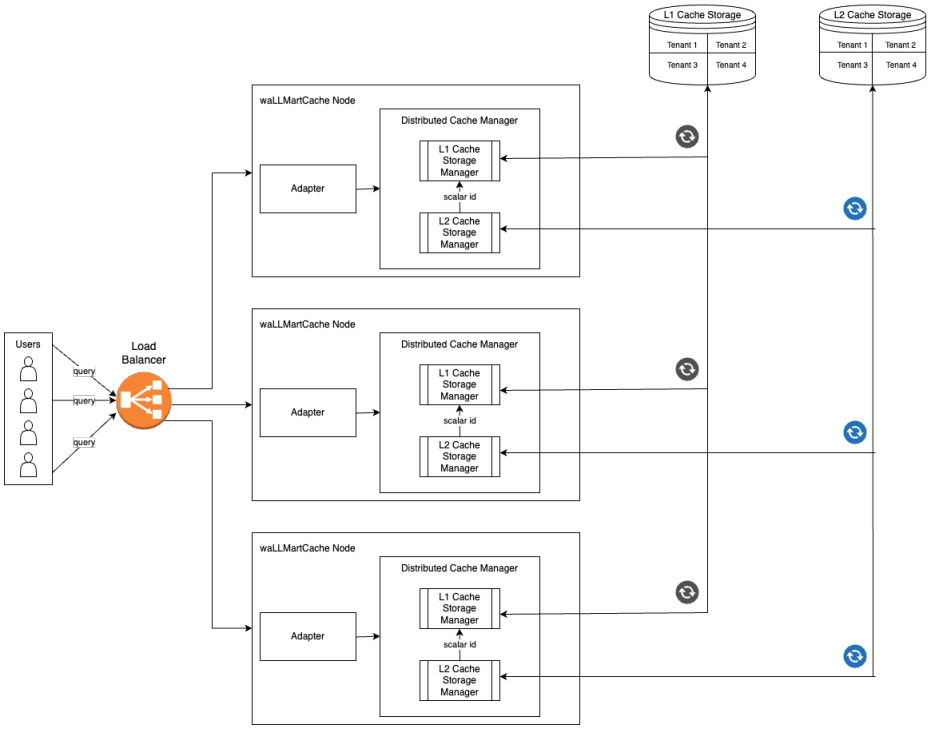

Fig. 2. Design of our distributed cache for LLMs. Note that we do not show each module from Fig. 1 here to maintain clarity and instead focus on the Distributed Cache Manager module specifically.

to be *erasable* based on the chosen eviction policy, e.g., LRU. Note that these deletions are carried out appropriately so that the two cache storages are in sync (albeit the synchronicity may not be achieved instantaneously but within a very small time lag in the order of milli-seconds), i.e., the vector embeddings and the LLM responses that are linked through the unique scalar ids are removed hand-in-hand.

4.3 Integrating Multi-Tenancy

One aspect that was not addressed by GPTCache at all was handling multiple tenants (i.e., clients) at the same time. Fundamentally, it is expected that the system should not return a response that originated from a query posed by a different client although it may be similar to the current query – returning such responses may reveal sensitive information and thus violate our privacy policies. To maintain the required semantic separations, we attach a metadata to the query that contains the tenant's id, and we maintain different instances of the L1 cache vector database corresponding to each tenant; upon receiving a new

query, we search for similar previous queries only in that vector database that belongs to the current tenant id. Conceptually, one may think that each database is partitioned into buckets based on tenants' ids as shown in Fig. 2.

4.4 Improved Decision Engine for Caching

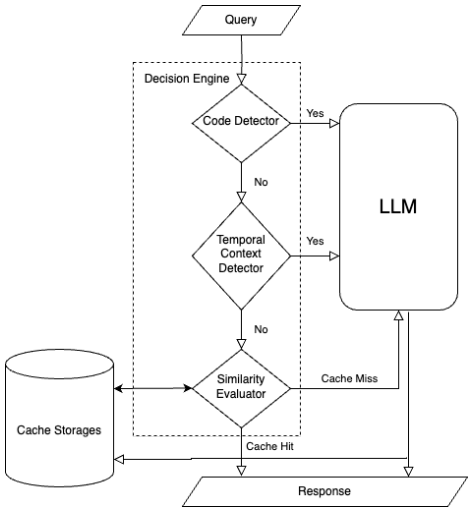

Fig. 3. Design of our decision engine which subsumes the Similarity Evaluator module of GPTCache.

Our *Decision Engine* module subsumes the *Similarity Evaluator* module from GPTCache. As illustrated in Fig. 3, the Decision Engine has three sub-modules that we elaborate upon in this subsection.

- **Code Detector:** This sub-module is used to detect whether the query being sent to the LLM contains code snippets or not. A user typically sends a code snippet to an LLM either to understand what functionality is achieved by the code or to seek the LLM's help in order to optimize it. However, during our experiments, we found that finding vector embedding based semantic similarity between two queries having code snippets is a notoriously difficult task that often leads to false positives, i.e., it results in erroneous cache hits. Two codes may differ by a single operator (say $+$ is replaced by $-$, or max by min) and thus produce different outputs – their vector embeddings however will be very similar thereby leading to fallacious cache hits. While such mistakes may happen to code-free texts as well, the number of false positives in such cases is distinctively low. Specifically, in our experiments, we found that true cache hits occurred 90% of the times for code-free queries whereas, the percentage of true cache hits for queries containing codes is only

9%; this number drops further to meagre 3% when the number of tokens for the code containing queries goes beyond 400. Therefore, we decided to include this sub-module and in case it finds that the query includes code, then we send it directly to the LLM and we do not cache such queries and corresponding responses.

- **Temporal Context Detector:** To understand the necessity of this sub-module, suppose we ask the LLM the question: *"What was yesterday's average temperature?"* Obviously, even if the *exact same question* is asked the next day, returning the cached answer would be wrong. Therefore, we do not cache those queries whose responses may differ with time. It is interesting to note that sometimes the responses may differ with time even if there is no temporal construct present in the query, for example, *"What is the price of item X?"* Businesses often employ *dynamic pricing* [8] because of which the price of item X may differ throughout the day, and hence caching the prices may not be advisable. Therefore, this sub-module is deployed with the relevant business intelligence baked in based on a specific client's needs.
- **Similarity Evaluator:** This sub-module is identical to that of GPTCache and already covered in Section 3.5.

It may be noted that the first two sub-modules in the Decision Engine have the positive side-effect of keeping the cache storages restrained.

4.5 Pre-loading (non-volatile) FAQs into the Cache

Table 2. Experimental results for finding the efficacy of pre-loaded FAQs

#Concurrent Users	#Requests	#RPS	Average Response Time (ms)	Median Response Time (ms)	P90 Response Time (ms)	Cache Hit w/o FAQ	Cache Hit w/ FAQ
1000	293304	329	45	35	70	80%	90%

LLMs are being integrated into chat bots. Chat bots are popularly used for handling queries related to *Customer Care* issues among others. Fortunately, many of the use-cases such as, Customer Care, often have to answer the same questions repeatedly. Such common questions and their responses can be collated into Frequently Asked Questions (FAQs) and can be pre-loaded into the cache to boost up its hits – we found this simple idea to be very effective, in fact, it improved the rate of cache hit by 10% as shown in Table 2. For the experiment whose results are reported in this table, we had turned the FAQs non-volatile, i.e., these were never evicted from the cache storages; however, making the pre-loaded FAQs non-volatile is not a mandatory requirement and these may be evicted in a similar fashion as other queries following an eviction policy. In our experience, we found that making the pre-loaded FAQs non-volatile yields superior performance.

Finally, we would like to mention that similar to making Redis available in the open source GPTCache repository, we have plans to open source our other contributions as well – we are currently waiting to receive the necessary approvals from the higher authorities.

4.6 Ablation Study

Table 3. Effects of Redis, distributed cache, decision engine and pre-loading FAQs on semantic caching

Method	**Correct Hit**			**Incorrect Hit**			**Correct Miss**			**Incorrect Miss**			**Acc(%)**	
	Reg	**Cde**	**Tmp**	**Reg**	**Cde**	**Tmp**	**Reg**	**Cde**	**Tmp**	**Reg**	**Cde**	**Tmp**	Reg	All
Oracle	500	500	0	0	0	0	600	600	1100	0	0	0	100	100
GPTCache	401	422	0	59	435	77	550	222	545	90	21	478	86.4	64.8
WMC(1N)	401	422	0	59	435	77	550	222	545	90	21	478	86.4	64.8
WMC(4N)	399	418	0	61	438	77	548	222	543	92	22	480	86.1	64.5
WMC(4N)+DE	399	0	0	61	0	0	548	600	1100	92	500	0	86.1	80.2
WMC(4N)+DE+FAQ	488	0	0	65	0	0	498	600	1100	49	500	0	89.6	81.4

Table 4. Effects of multi-tenancy on semantic caching

Method	**T**	**Correct Hit**			**Incorrect Hit**			**Correct Miss**			**Incorrect Miss**			**Acc(%)**	
		Reg	**Cde**	**Tmp**	**Reg**	**Cde**	**Tmp**	**Reg**	**Cde**	**Tmp**	**Reg**	**Cde**	**Tmp**	**Reg**	**All**
Oracle	1	250	250	0	0	0	0	300	300	550	0	0	0	100	100
	2	250	250	0	0	0	0	300	250	550	0	0	0		
GPTCache	1	199	215	0	55	288	59	250	36	245	46	11	246	82	57
	2	202	207	0	52	291	61	251	36	240	45	16	249		
WMC(4N)+DE+FAQ	1	211	0	0	55	0	0	249	300	550	35	250	0	83.8	79.4
	2	212	0	0	54	0	0	250	300	550	34	250	0		
WMC(4N)+DE+FAQ+MT	1	244	0	0	33	0	0	247	300	550	26	250	0	89.5	81.3
	2	244	0	0	32	0	0	249	300	550	25	250	0		

In this subsection, we explore what are the effects of the various constituents of waLLMartCache with respect to GPTCache, and an Oracle that makes no incorrect cache hit or miss – this Oracle is used to benchmark the performance of GPTCache and our waLLMartCache. For these experiments, we chose a set of 100 prompts containing code (Cde), 100 textual prompts containing temporal context (Tmp) and 100 (regular) textual prompts without any temporal context (Reg). For each prompt we create 5 semantically similar prompts (which should ideally result in correct cache hits) and 5 dissimilar prompts (which should ideally result in correct cache misses) – note that the 5 dissimilar prompts need to be pairwise semantically different as well to ensure cache misses. We use GPT-4 [21] to generate the semantically similar and dissimilar prompts from a given textual prompt. For generating similar prompts, we instruct GPT-4 to make changes such as, active to passive voice, compound to multiple simple sentences, positive

to double negative (e.g., "present" to "not absent"), replace a single or multiple words by their synonyms; similarly, for producing dissimilar prompts, we make changes such as, positive to negative, replace a single or multiple words by their antonyms, replace the entire prompt by some random unrelated Wikipedia sentence(s). We have manually checked whether the resulting prompts were indeed similar or dissimilar. In case of prompts containing code, to generate prompts with similar meaning, we replaced equations by their mathematical equivalent ones (e.g., $y + y$ to $2 * y$), or added some constant and later subtracted the same constant, etc. To produce dissimilar prompts with code, we deliberately changed some of the operators, or removed equations partially or totally. Thus, each prompt led to 10 additional prompts, and overall we had 3300 prompts (including the original ones). The entire set is randomly shuffled before invoking the LLM caches.

Table 3 underlines the effects of Redis, distributed cache, decision engine and pre-loading FAQs (i.e., all the constituents except multi-tenancy) on waLLMart-Cache (WMC). Note that we report the correct and the incorrect cache hits and misses similar to earlier literature [5, 26]; we further show the break ups for all three types of prompts: Reg, Cde and Tmp. To the best of our knowledge, all related previous works [5, 33, 26, 14, 18] only focused on Reg(ular) textual prompts, and did not delve into prompts containing codes or having a specific type of context such as, temporal in our case, that may require specific caching strategies. Accordingly, we report accuracy (Acc) for Reg prompts only (to maintain consistency with earlier literature) along with all types of prompts together (All). For retail business use-cases, typically prompts having temporal context should not be cached, for example, when dynamic pricing [8] is employed. Therefore, as per our strategy, all textual prompts having temporal context should result in cache miss; consequently, the Oracle in Table 3 has zero correct hits and 1100 correct misses for Tmp category of prompts. For the other two categories, the ideal number of correct cache hits is 500 (because of similar prompts) while the ideal number of correct cache misses is 600 (because of dissimilar prompts and the original prompts which should result in misses when executed for the first time). All experiments involving GPTCache and waLLMartCache were carried out five times – here we report their means rounded to nearest integer. In Table 3, WMC(1N) represents waLLMartCache deployed to only a single node – in this case, it differs from GPTCache only in its use of Redis as L2 storage. As expected, there is practically no difference between GPTCache and WCM(1N) on semantic caching because these have the same Similarity Evaluator although Redis contributes in reaching much higher RPS as mentioned in Section 4.1. waLLMartCache distributed to four nodes is represented as WCM(4N) which sees negligible drop in accuracies but it aids in scaling the system drastically and boosts RPS further. However, when it is augmented with the decision engine (WCM(4N)+DE), the DE forces the cache hits (both correct and incorrect) to zero for Cde and Tmp categories of prompts. Since ideally all Tmp prompts should result in cache misses, the positive effect of DE is obvious; in case of Cde prompts, we found DE to be beneficial because although the Similarity Eval-

uator helped in getting some correct cache hits, it produced larger number of incorrect cache hits thus bringing the overall accuracy down. Therefore, although the accuracy for Reg remains identical for WCM(4N)+DE, the accuracy for All improves considerably. It may be noteworthy that from the results of Table 3, it may seem that our DE can identify codes and temporal contexts perfectly from a given text; however, the designed DE may need further enhancements especially in case of identifying temporal contexts. We believe that identifying code snippets is not difficult if one builds a dictionary of all the keywords supported by the programming languages of interest along with looking for curly braces, sentences ending with semi-colon, etc. However, identifying temporal context may be challenging because humans sometimes write English in convoluted fashions – the 100 Tmp prompts chosen for this experiment were not too complex and our DE could identify these in all cases even for their similar and dissimilar counterparts. Finally, pre-loading FAQs has a beneficial effect because now potentially even the first executions of the prompts can get a correct cache hit if it is part of the FAQ list as can be seen from the additional improvements in the accuracies for both Reg and, in turn, for All. Note that, in practice, the advantage of having a pre-loaded FAQ can be even more based on the length of the FAQ and the frequency in which these prompts are repeated as evidenced in Section 4.5.

To measure the effect of supporting multi-tenancy, we distribute the prompts from the earlier experiments to two equal halves and assign these to Tenant (T) 1 and 2 – the results are captured in Table 4. Since GPTCache and WMC(4N)+DE +FAQ cannot distinguish between prompts from the different tenants, the number of incorrect cache hits is proportionately much higher compared to the previous scenario. We refrain from mentioning the numbers for WMC(1N), WMC(4N) and WMC(4N)+DE in Table 4 for brevity and also because we believe that these numbers can be derived from the other rows in this table. Lastly, WMC(4N)+DE+FAQ+MT which has multi-tenancy support is able to do the necessary distinction and is able to achieve accuracies similar to that of WMC(4N)+DE+FAQ in Table 3 which did not have multiple tenants and therefore, their accuracies should be equivalent. Usually, the positive effect of multi-tenancy increases linearly with the increase in the number of tenants.

5 Conclusion

LLMs have recently had profound effect on our lives. Programming, story telling, reporting, auditing, chatting – all these activities can possibly be enhanced by many folds by using LLMs effectively. Consequently, all industries are presently in a race to leverage this new AI technology to increase their customer experiences and revenues as quickly as possible. However, LLMs can be quite costly and for large industries their costs can easily shoot up given their massive requirements. Furthermore, too many invocations of the proprietary LLMs often lead to service outages which are resumed only after some cool-down period. Therefore, caching the responses of the LLMs can be an effective way to not only reduce the costs but ensure better availability of the services. Additionally,

returning cached responses can reduce the latency significantly and provide a more stable system. In [5], the authors develop a caching mechanism for LLMs called GPTCache. However, when we tried to use this tool off-the-shelf, we found it to be lacking in various aspects that made its adoption in an industrial setting difficult. Accordingly, we enhanced this tool and developed an in-house LLM caching system called waLLMartCache that can additionally (i) support Redis database (this contribution of ours is already integrated with the open source GPTCache), (ii) scale to multiple nodes with support for a distributed eviction manager, (iii) handle multiple individual tenants while maintaining semantic separations, (iv) handle better semantic caching based on our decision engine, and (v) pre-load the cache with (non-volatile) FAQs, if needed, that boost cache hits significantly. In future, we plan to incorporate more LLMs and vector databases in our system, and explore alternate cache eviction policies. We also plan to check if switching LLMs in the interim based on historical data is a good idea or not, i.e., use a more powerful LLM initially so that our cache is populated with richer responses and then transition to a less powerful LLM if we believe that most responses in the foreseeable future will be returned from the cache.

References

1. Agarwal, S., Mitra, S., Chakraborty, S., Karanam, S., Mukherjee, K., Saini, S.K.: Approximate caching for efficiently serving text-to-image diffusion models. In: USENIX NSDI (2024)
2. Almazrouei, E., Alobeidli, H., Alshamsi, A., Cappelli, A., Cojocaru, R., Debbah, M., Goffinet, É., Hesslow, D., Launay, J., Malartic, Q., Mazzotta, D., Noune, B., Pannier, B., Penedo, G.: The falcon series of open language models. CoRR **abs/2311.16867** (2023)
3. Anil, R., Borgeaud, S., Wu, Y., Alayrac, J., Yu, J., Soricut, R., Schalkwyk, J., Dai, A.M., Hauth, A., Millican, K., Silver, D., Petrov, S., Johnson, M., Antonoglou, I., Schrittwieser, J., Glaese, A., Chen, J., Pitler, E., Lillicrap, T.P., Lazaridou, A., Firat, O., Molloy, J., Isard, M., Barham, P.R., Hennigan, T., Lee, B., Viola, F., Reynolds, M., Xu, Y., Doherty, R., Collins, E., Meyer, C., Rutherford, E., Moreira, E., Ayoub, K., Goel, M., Tucker, G., Piqueras, E., Krikun, M., Barr, I., Savinov, N., Danihelka, I., Roelofs, B., White, A., Andreassen, A., von Glehn, T., Yagati, L., Kazemi, M., Gonzalez, L., Khalman, M., Sygnowski, J., et al.: Gemini: A family of highly capable multimodal models. CoRR **abs/2312.11805** (2023)
4. Anthropic: Introducing Claude. https://www.anthropic.com/news/introducing-claude, accessed: 2024-04-05
5. Bang, F.: GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings. In: Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023). pp. 212–218. Association for Computational Linguistics (2023). https://doi.org/10.18653/v1/2023.nlposs-1.24
6. Barnett, S., Kurniawan, S., Thudumu, S., Brannelly, Z., Abdelrazek, M.: Seven failure points when engineering a retrieval augmented generation system. In: CAIN. pp. 194–199 (2024)
7. Bengio, Y., Ducharme, R., Vincent, P.: A neural probabilistic language model. In: NeurIPS (2000)

8. Bertsimas, D., Perakis, G.: Dynamic Pricing: A Learning Approach, pp. 45–79. Springer US (2006)
9. Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D.M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., Amodei, D.: Language models are few-shot learners. In: NeurIPS (2020)
10. Cheng, K., Hu, W., Wang, Z., Du, P., Li, J., Zhang, S.: Enabling efficient batch serving for lmaas via generation length prediction. CoRR **abs/2406.04785** (2024)
11. Dar, S., Franklin, M.J., Jónsson, B.Þ., Srivastava, D., Tan, M.: Semantic data caching and replacement. In: VLDB. pp. 330–341 (1996)
12. Databricks: Dolly. https://github.com/databrickslabs/dolly, accessed: 2024-04-05
13. Gallegos, I.O., Rossi, R.A., Barrow, J., Tanjim, M.M., Kim, S., Dernoncourt, F., Yu, T., Zhang, R., Ahmed, N.K.: Bias and fairness in large language models: A survey. CoRR **abs/2309.00770** (2023)
14. Gill, W., Elidrisi, M., Kalapatapu, P., Anwar, A., Gulzar, M.A.: Privacy-aware semantic cache for large language models. CoRR **abs/2403.02694** (2024)
15. Handy, J.: The cache memory book. Academic Press Professional, Inc. (1993)
16. Jiang, A.Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D.S., de Las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L.R., Lachaux, M., Stock, P., Scao, T.L., Lavril, T., Wang, T., Lacroix, T., Sayed, W.E.: Mistral 7b. CoRR **abs/2310.06825** (2023)
17. Lee, D., Chu, W.W.: Semantic caching via query matching for web sources. In: CIKM. pp. 77–85 (1999)
18. Li, J., Xu, C., Wang, F., von Riedemann, I.M., Zhang, C., Liu, J.: SCALM: Towards semantic caching for automated chat services with large language models. CoRR **abs/2406.00025** (2024)
19. Mazmudar, M., Humphries, T., Liu, J., Rafuse, M., He, X.: Cache me if you can: Accuracy-aware inference engine for differentially private data exploration. Proc. VLDB Endow. **16**(4), 574–586 (2022)
20. OpenAI: Pricing. https://openai.com/pricing, accessed: 2024-04-04
21. OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F.L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., Avila, R., Babuschkin, I., Balaji, S., Balcom, V., Baltescu, P., Bao, H., Bavarian, M., Belgum, J., Bello, I., Berdine, J., Bernadett-Shapiro, G., Berner, C., Bogdonoff, L., Boiko, O., Boyd, M., Brakman, A.L., Brockman, G., Brooks, T., Brundage, M., Button, K., Cai, T., Campbell, R., Cann, A., Carey, B., Carlson, C., Carmichael, R., Chan, B., Chang, C., Chantzis, F., Chen, D., Chen, S., Chen, R., Chen, J., Chen, M., Chess, B., Cho, C., Chu, C., Chung, H.W., Cummings, D., Currier, J., Dai, Y., Decareaux, C., Degry, T., Deutsch, N., Deville, D., Dhar, A., Dohan, D., Dowling, S., Dunning, S., Ecoffet, A., Eleti, A., Eloundou, T., Farhi, D., Fedus, L., Felix, N., Fishman, S.P., Forte, J., Fulford, I., Gao, L., Georges, E., Gibson, C., Goel, V., Gogineni, T., Goh, G., Gontijo-Lopes, R., Gordon, J., Grafstein, M., Gray, S., Greene, R., Gross, J., Gu, S.S., Guo, Y., Hallacy, C., Han, J., Harris, J., He, Y., Heaton, M., Heidecke, J., Hesse, C., Hickey, A., Hickey, W., Hoeschele, P., Houghton, B., Hsu, K., Hu, S., Hu, X., Huizinga, J., Jain, S., Jain, S., Jang, J., Jiang, A., Jiang, R., Jin, H., Jin, D., Jomoto, S., Jonn, B., Jun, H., Kaftan, T., Łukasz Kaiser, Kamali, A., Kanitscheider, I., Keskar, N.S., Khan, T., Kilpatrick, L., Kim, J.W., Kim, C., Kim, Y., Kirchner, J.H., Kiros, J., Knight, M., Kokotajlo, D., Łukasz Kondraciuk,

Kondrich, A., Konstantinidis, A., Kosic, K., Krueger, G., Kuo, V., Lampe, M., Lan, I., Lee, T., Leike, J., Leung, J., Levy, D., Li, C.M., Lim, R., Lin, M., Lin, S., Litwin, M., Lopez, T., Lowe, R., Lue, P., Makanju, A., Malfacini, K., Manning, S., Markov, T., Markovski, Y., Martin, B., Mayer, K., Mayne, A., McGrew, B., McKinney, S.M., McLeavey, C., McMillan, P., McNeil, J., Medina, D., Mehta, A., Menick, J., Metz, L., Mishchenko, A., Mishkin, P., Monaco, V., Morikawa, E., Mossing, D., Mu, T., Murati, M., Murk, O., Mély, D., Nair, A., Nakano, R., Nayak, R., Neelakantan, A., Ngo, R., Noh, H., Ouyang, L., O'Keefe, C., Pachocki, J., Paino, A., Palermo, J., Pantuliano, A., Parascandolo, G., Parish, J., Parparita, E., Passos, A., Pavlov, M., Peng, A., Perelman, A., de Avila Belbute Peres, F., Petrov, M.: Gpt-4 technical report (2024)
22. Pinecone: Build knowledgeable ai. https://www.pinecone.io/, accessed: 2024-04-08
23. Qdrant: qdrant. https://github.com/qdrant/qdrant, accessed: 2024-04-08
24. Qiu, H., Mao, W., Patke, A., Cui, S., Jha, S., Wang, C., Franke, H., Kalbarczyk, Z.T., Basar, T., Iyer, R.K.: Efficient interactive LLM serving with proxy model-based sequence length prediction. CoRR **abs/2404.08509** (2024)
25. Qiu, H., Mao, W., Patke, A., Cui, S., Jha, S., Wang, C., Franke, H., Kalbarczyk, Z.T., Basar, T., Iyer, R.K.: Power-aware deep learning model serving with μ-serve. In: USENIX ATC (2024)
26. Rasool, Z., Barnett, S., Willie, D., Kurniawan, S., Balugo, S., Thudumu, S., Abdelrazek, M.: Llms for test input generation for semantic applications. In: CAIN. pp. 160–165 (2024)
27. Redis: Get the world's fastest in-memory database from the ones who built it. https://redis.io/, accessed: 2024-04-09
28. Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Canton-Ferrer, C., Chen, M., Cucurull, G., Esiobu, D., Fernandes, J., Fu, J., Fu, W., Fuller, B., Gao, C., Goswami, V., Goyal, N., Hartshorn, A., Hosseini, S., Hou, R., Inan, H., Kardas, M., Kerkez, V., Khabsa, M., Kloumann, I., Korenev, A., Koura, P.S., Lachaux, M., Lavril, T., Lee, J., Liskovich, D., Lu, Y., Mao, Y., Martinet, X., Mihaylov, T., Mishra, P., Molybog, I., Nie, Y., Poulton, A., Reizenstein, J., Rungta, R., Saladi, K., Schelten, A., Silva, R., Smith, E.M., Subramanian, R., Tan, X.E., Tang, B., Taylor, R., Williams, A., Kuan, J.X., Xu, P., Yan, Z., Zarov, I., Zhang, Y., Fan, A., Kambadur, M., Narang, S., Rodriguez, A., Stojnic, R., Edunov, S., Scialom, T.: Llama 2: Open foundation and fine-tuned chat models. CoRR **abs/2307.09288** (2023)
29. Wang, J., Yi, X., Guo, R., Jin, H., Xu, P., Li, S., Wang, X., Guo, X., Li, C., Xu, X., Yu, K., Yuan, Y., Zou, Y., Long, J., Cai, Y., Li, Z., Zhang, Z., Mo, Y., Gu, J., Jiang, R., Wei, Y., Xie, C.: Milvus: A purpose-built vector data management system. In: SIGMOD. pp. 2614–2627 (2021)
30. Weaviate: Welcome to weaviate docs. https://weaviate.io/developers/weaviate, accessed: 2024-04-08
31. Xu, D., Zhang, D., Yang, G., Yang, B., Xu, S., Zheng, L., Liang, C.: Survey for landing generative ai in social and e-commerce recsys – the industry perspectives. CoRR **abs/2406.06475** (2024)
32. Ye, J., Chen, X., Xu, N., Zu, C., Shao, Z., Liu, S., Cui, Y., Zhou, Z., Gong, C., Shen, Y., Zhou, J., Chen, S., Gui, T., Zhang, Q., Huang, X.: A comprehensive capability analysis of GPT-3 and GPT-3.5 series models. CoRR **abs/2303.10420** (2023)
33. Zhu, H., Zhu, B., Jiao, J.: Efficient prompt caching via embedding similarity. CoRR **abs/2402.01173** (2024)